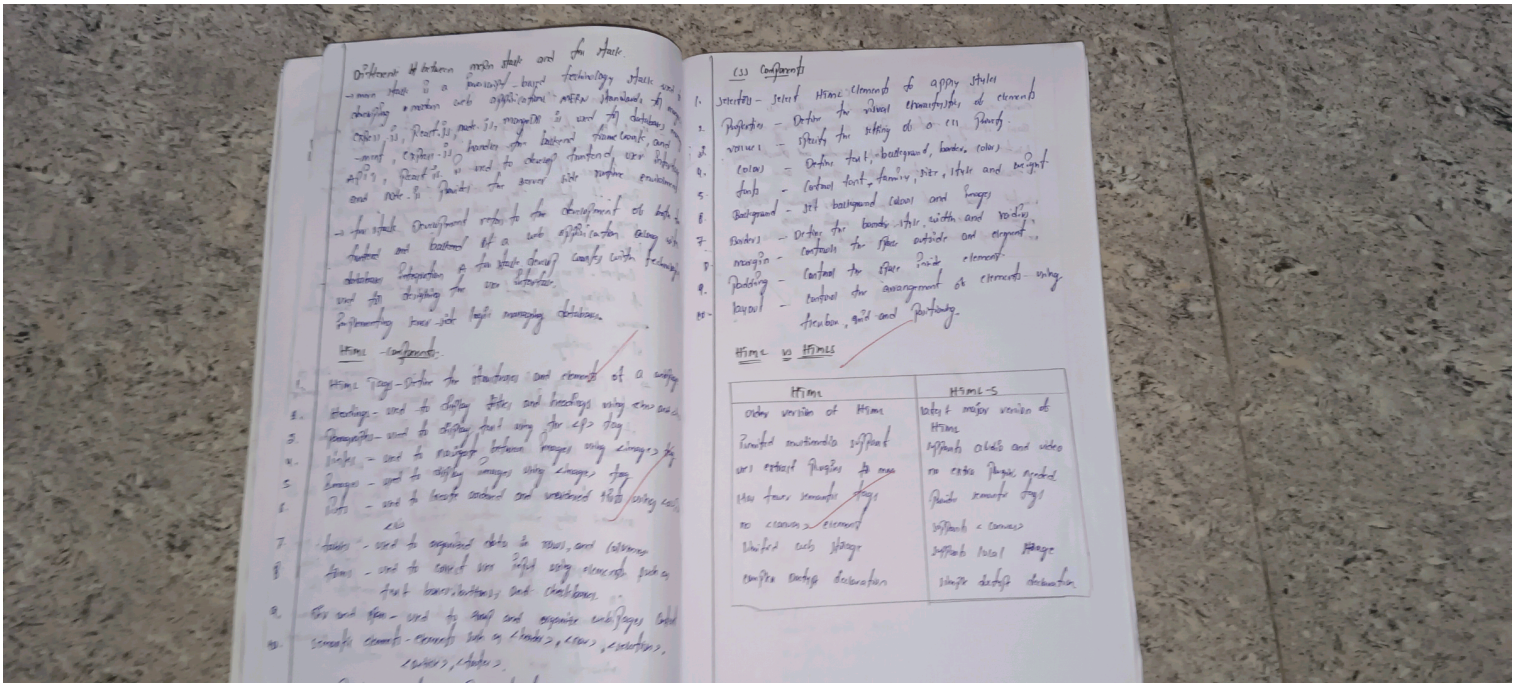
Introduction to HTML and CSS.

→ HTML (Hyper text markup language) is the standard markup language used to create and structure content on web pages. it defines the basic structure of a web page. it defines the basic structure of a web page using elements and tags such as headings, paragraphs, links, images, lists, tables, table forms, and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (Cascading Style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML document it is used to define properties such as colors, fonts, background, spacing, borders, alignment, dimensions, and layouts, CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices, CSS can be applied using inline, internal external stylesheets

→ HTML and CSS work together to develop webpages, and where HTML provides the structure and content, where CSS provides the styling, layout, visual, presentation. The combination of both forms the foundation of modern web development and enables the creation of organized attractive and user friendly web interfaces.



Difference between main stack and the stack.

→ main stack is a javascript - based technology stack and ecosystem used for building web applications. MERN stands for MongoDB, Express.js, React.js, and Node.js, each of which is used for different purposes in the stack.

MongoDB is used for storing data in a flexible, document-based format. Express.js is used for building the backend of the application, including handling HTTP requests and responses, routing, and middleware. React.js is used for building the user interface of the application, including components, state management, and user interactions. Node.js is used for running the server-side JavaScript code that powers the backend of the application.

→ The stack development refers to the development of both the frontend and backend of a web application using the MERN stack. It involves integrating the various technologies in the stack to create a fully functional web application with features such as user authentication, data storage and retrieval, and dynamic user interfaces.

Implementing token-based authentication using JSON Web Tokens (JWTs) for user authentication and authorization.

HTML - components

1. HTML Page - Define the structure and elements of a webpage
2. Headings - used to display titles and headings using <h1> to <h6> tags
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between pages using <a> tag
5. Images - used to display images using <img> tag
6. Lists - used to create ordered and unordered lists using <ol> and
7. Tables - used to organize data in rows and columns
8. Forms - used to collect user input using elements such as text boxes, buttons, and checkboxes
9. Div and Span - used to group and organize webpages content
10. Semantic elements - elements such as <header>, <nav>, <section>, <article>, <footer>

CSS Components

1. Selectors - Select HTML elements to apply styles
2. Properties - Define the visual characteristics of elements
3. Values - Specify the setting of a CSS property
4. Color - Define text, background, border, color
5. Fonts - Control font family, size, style and weight
6. Background - Set background color and images
7. Borders - Define the border style, width and radius
8. Margin - Controls the space outside and element
9. Padding - Control the space inside element
10. Layout - Control the arrangement of elements using flexbox, grid and positioning

HTML vs HTML5

HTML	HTML-5
Older version of HTML	Latest major version of HTML
Provided functionality different	Supports video and audio
Use external plugins to run	No extra plugin needed
Have fewer semantic tags	Provide semantic tags
No <canvas> element	Supports <canvas>
Limited cache storage	Supports local storage
Complex doctype declaration	Simple doctype declaration

Difference between mern stack and full stack.

→ mern stack is a javascript-based technology stack used for developing modern web applications. MERN stands for mongoDB, express.js, React.js, node.js, mongoDB is used for database management, express.js handles the backend framework, and APIs, React.js is used to develop frontend, user interface and node.js handles the server side runtime environment.

→ full stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic managing databases.

HTML - components:

1. HTML Tags - Define the structures and elements of a webpage
2. Headings - used to display titles and headings using <h1> and <h2>
3. Paragraphs - used to display text using the <p> tag.
4. Links - used to navigate between pages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organized data in rows and columns
8. Forms - used to collect user input using elements such as text boxes, buttons, and checkboxes.
9. Div and span - used to group and organize webpages content
10. Semantic elements - elements such as <header>, <nav>, <section>, <article>, <footer>.

Provide meaningful webpages structure

(3) Components

1. Selectors - Select HTML elements to apply styles
2. Properties - Define the visual characteristics of elements
3. values - specify the setting of a CSS property.
4. Color - Define text, background, border color)
5. fonts - Control font, family, size, style and weight
6. Background - set background colour and images
7. Borders - Define the border style, width and radius
8. margin - controls the space outside and element.
9. Padding - Control the space inside element.
10. layout - Control the arrangement of elements using flexbox, grid and positioning.

HTML vs HTML5

HTML	HTML-5
older version of HTML	latest major version of HTML
Limited multimedia support	supports audio and video
use extract plugins to run	no extra plugins needed
Has fewer semantic tags	Provides semantic tags
no <canvas> element	supports <canvas>
limited web storage	supports local storage
complex doctype declaration	simple doctype declaration

CSS	CSS3
Basic version of CSS	Advanced version of CSS
Basic styling features	Provides advanced styling features
Limited animations	Supports animations and transitions
Not flexible layout system	Supports flexible and grid
Limited visual effects	Supports shadows, gradient effects
No media queries	Supports media queries
Use a high specificity	Avoids high specificity

Design a static web page using HTML and CSS

Parmeshwar Tiwari, Entry No. 23

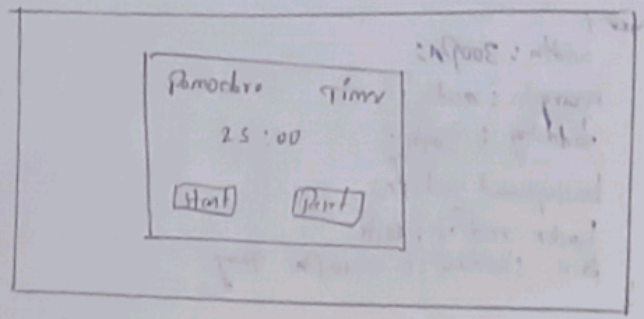
```
<!doctype html>
<html>
 <head>
  <title> Parmeshwar Tiwari </title>
  <style>
   body {
     font-family: Arial;
     background: #F5F5F5;
     text-align: center;
     padding: 50px;
   }
  .timer {
     width: 300px;
     margin: auto;
     padding: 30px;
     background: white;
     border-radius: 15px;
     box-shadow: 0 0 10px gray;
   }
  h1 {
   color: #ff6C3C;
  }
  .timer {
     font-size: 50px;
     margin: 20px;
  }
  button {
     padding: 10px 20px;
     margin: 5px;
     border: none;
     border-radius: 5px;
     background: #E74C1C;
```

color: white;
}
</style>
</head>
<body>

<h1> Pomodoro timer</h1>
25:00
<button> start </button>
<button> Reset </button>

</body>
</html>

output:



(i). Design an HTML code using audio tags

Code:-

```
<!DocType Html>
<html>
<head>
  <title> Audio tags </title>
</head>
<body>
    <h2> my favorite song </h2>
    <audio controls>
      <source src="song.mp3" type="audio/mpeg">
      your browser does not support the audio tag.
    </audio>
</body>
</html>
```

O/P:- Displays an audio player with play, pause and volume controls.

Steps:-

Create a new HTML file
Add the basic HTML structure
Insert the <audio> tag with controls attribute
Specify the audio file using <source> tag.
Save and open the HTML file in a web browser
Play the audio using displayed controls.

(i) Design a Html code using video tags.

code:

```
<! Doctype html>
<html>
<head>
   <title> video example </title>
</head>
<body>
   <h2> my video </h2>
   <video width="400" controls>
      <source src="video.mp4" type="video/mp4">
   </video>
</body>
</html>
```

o/p:- Display a video player with play, pause, volume and full screen controls.

steps:-

1. create a new Html file
2. add a basic Html structure
3. add a heading using the <h1> tag
4. insert the <video> tag with controls attribute
5. specify the video file using <source> tag
6. save the file and open it in browser
7. play the video using displayed controls.

write a program using javascript (arrays, functions)

code:

```
let numbers = [12, 45, 8, 23, 50, 19, 34, 67, 90, 11];
function displayArray(arr) {
    console.log("Array elements:");
    for (let i=0; i<arr.length; i++) {
        console.log(arr[i]);
    }
}
function findSum(arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++)
    {
        sum = arr[i];
    }
    return sum;
}
```

Experiment - 04

Develop dynamic webpages using JavaScript.

Program:-

```
<!Doctype html>
<html>
<head>
<title> Dynamic webpage </title>
<style>
    body {
        text_align : center;
        font_family = Arial;
        margin_top : 100px;
    }
    button {
        padding : 10px, 20px;
        font_size : 16px;
        cursor : pointer;
    }
</style>
</head>
<body>
    <h1 id="heading"> welcome to my webpage </h1>
    <p id="message"> Click the button to change
                     this page </p>
    <button onclick="changepage()"> Click me </button>
    <script>
        function changepage() {
            document.getElementById("heading").
            innerHtml = "welcome to javascript";
            document.getElementById("message").innerHtml = "The
            webpage has been changed dynamically";
            document.body.style.backgroundColor = "lightblue";
        }
    </script>
</body>
</html>
```

Output:-

welcome to my webpage

Click the button to change this web page

Click me

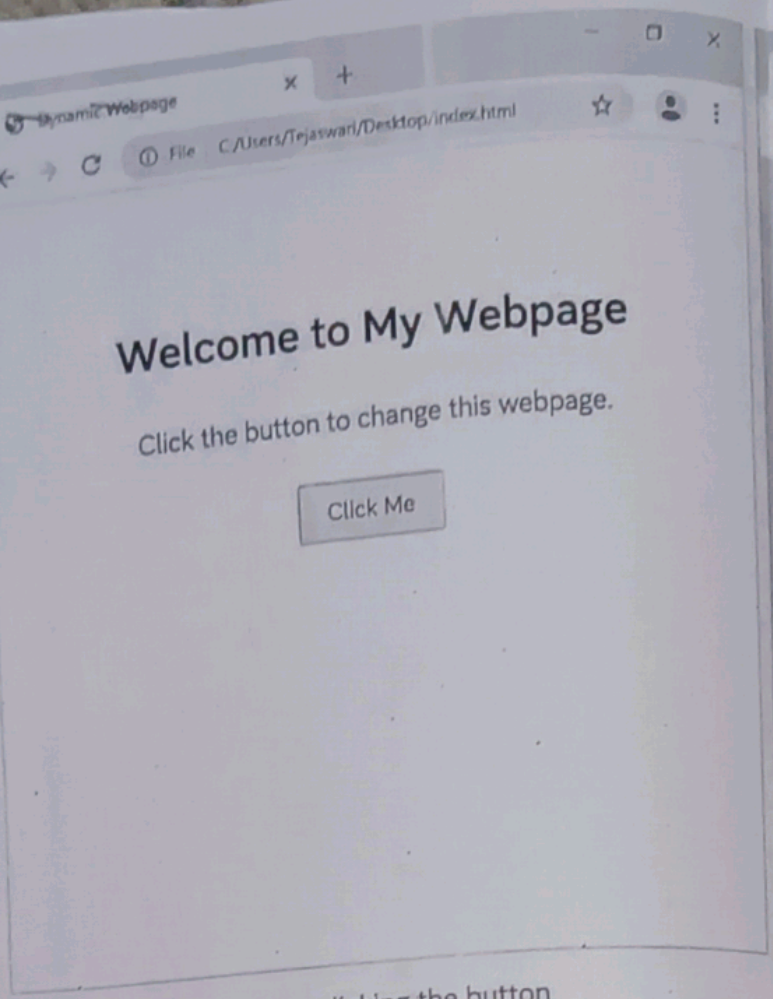
When user clicks 'click me' the javascript function changepage is called, then the heading dynamically changed and background also changes

After clicking the button:-

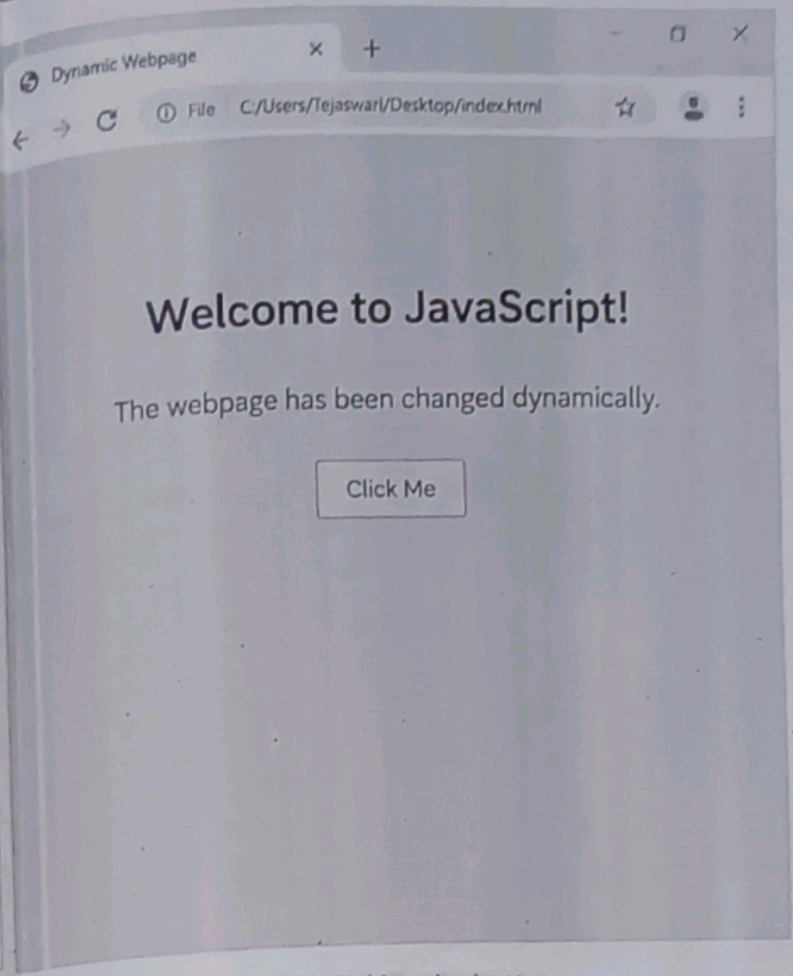
welcome to javascript!

This page has been dynamically changed

Click me



Before clicking the button



After clicking the button

output : 15

3
5

Ex Experiment - 5

Implement modules in node JS

In node JS, you can implement modules using two distinct systems: common JS (CJS), which is traditional default and is node (ESM), the modern browser standard

1. The common JS method (default);

common JS use modules exports to share code and require() to import it this system load modules synchronization

(a) Create the module (math.js)

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract };
```

(b) use the module (app.js)

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

output : JS

```
15
5
```

Ex 6 Experiment - 5

Implement modules in node.js

In node.js, you can implement modules using two distinct systems: common.js (cjs), which is traditional default and is module (esm), the modern based script standard

1. The common.js method (default):

common.js uses modules exports to share code and require() to import it. This system load modules synchronization.

(a) create the module (math.js)

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract };
```

(b) use the module (app.js)

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

output: js

5

2. The ES modules method (modern standard)

ES modules use export to share code and import to use it. To make node.js recognize ES modules, you must either use the .mjs file extension or add this: "module" to your package.json file.

a. Create the module (calculator).js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculator name = "super calc";
export default calculator name;
```

b. use the module (main.js)

```
import calculator name = {multiply, divide}
    from "./calculator.js";
console.log(calculator name);
console.log(multiply(4,2));
console.log(divide(4,2));
```

output:

```
super calc
8
2
```

using built in core modules:

node.js comes equipped with dozens of built-in core modules (like fs for file system or path for file routing) that don't need installation we can load them by

module names:

```
* // common js style
    const fs = require("fs")

// ES modules style
    import fs from 'fs';
```

Here are some of most common ones:

1. fs = file system operations.
2. http = http server and client.
3. path = file path utils
4. os = operating system utils
5. events = event handling
6. util = utility functions
7. stream = stream handling
8. crypto = cryptographic functions.
9. url = url parsing
10. querystring = url query string handling.

example simple http server.

```
http.create server((req, res) => {
        res.writeHead(200, {'content-type': 'text
    res.end('Hello world!!'); //Hello World
});
listen(3000);
```

2. The ES module method (modern standard)

ES modules use export to share code and import to use it. To make node.js recognize ES modules, you must either use the .mjs extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js)

```
export const multiply = (a, b) => a * b;
export const divide = (a, b) => a / b;
const calculator name = 'super calc';
export default calculator name;
```

b. Use the module (main.js)

```
import calculator, { multiply, divide } from
    from './calculator.js';
console.log(calculator name);
console.log(multiply(4, 2));
console.log(divide(8, 2));
```

output:

```
super calc
8
2
```

using built in Core modules:

Node.js comes equipped with dozens of built-in core modules (like fs for file system, http for networking) that don't need to be installed. You load them by name:

```
// Import the http module
const http = require('http');

// Import the fs module
const fs = require('fs');
```

Here are some of the most commonly used:

1. fs = file system operations
2. http = http server and client
3. path = file path utilities
4. os = operating system utilities
5. crypto = security hashing
6. util = utility functions
7. stream = stream handling
8. crypto - cryptographic functions
9. url - url parsing
10. querystring - url query string handling

example: simple http server

```
http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Hello World!');
}).listen(3000);
```

Experiment-6

Develop node application using expressjs and node js
simple node application using node js and express js

Student management system

1. add a student
2. view all students
3. delete a student
4. use express js router
5. simple html interface.

Project structure:

student app/
- server.js
- public
- index.html

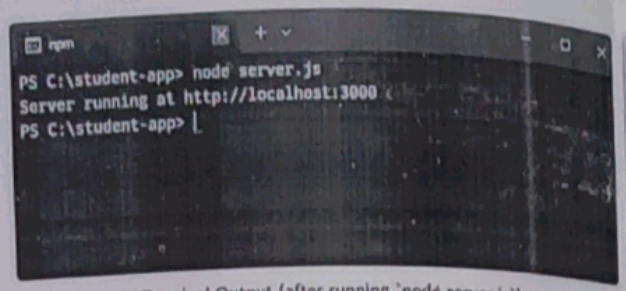
Server.js

```
const express = require("express");
const app = express();
const port = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
    res.json(students);
});
// Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "Student added successfully", student: student });
});
// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student => student.id !== id);
    res.json({
        message: "Student deleted successfully"
    });
});
// Start server
app.listen(port, () => {
    console.log("Server running at http://localhost:" + port);
});
```

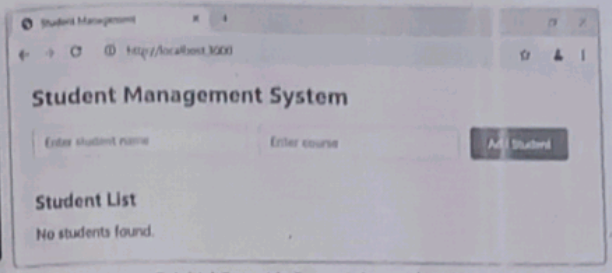
public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student management</title>
</head>
<body>
```

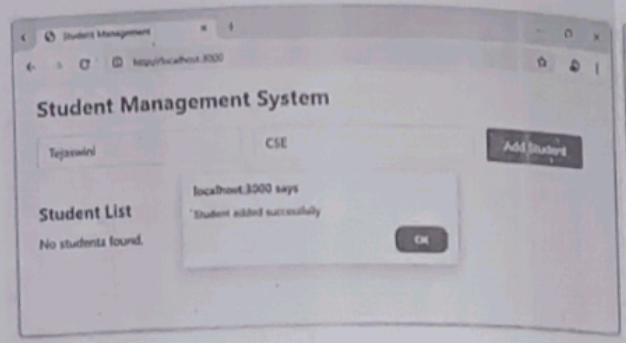
```
<h1> Student management </h1>
<input type="text" id="name" placeholder="enter
    Student name">
<input type="text" id="count" placeholder="enter count">
<button onclick="addStudent()"> Add Student </button>
<h2> Student list </h2>
<ul id="studentlist"></ul>
<script>
    function addStudent() {
        let name = document.getElementById("name").
        let count = document.getElementById("count").
                                    value;
        fetch("/students", {
            method: "post",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({
                name: name,
                count: count,
            })
        })
        .then(response => response.json())
        .then(data => {
            alert(data.message);
            document.getElementById("name").value = "";
            document.getElementById("count").value = "";
            display.Students();
        });
    }
    function displayStudents() {
        fetch("/students").then(response => response.json())
        .then(students => {
            let list = document.getElementById("studentlist");
            list.innerHtml = "";
            students.forEach(student => {
                let li =
                document.createElement("li");
                li.innerHtml = student.id + ". " +
                        student.name + " - " +
                        student.count + " <button onclick=
                "deleteStudent(${student.id})">Delete</button>";
                list.appendChild(li);
            });
        });
    }
    function deleteStudent(id) {
        fetch("/students/" + id, {method: "DELETE"})
        .then(response => response.json())
        .then(data => {
```



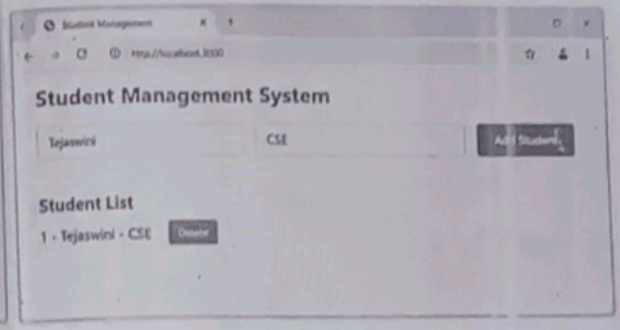
1. Terminal Output (after running `node server.js`)



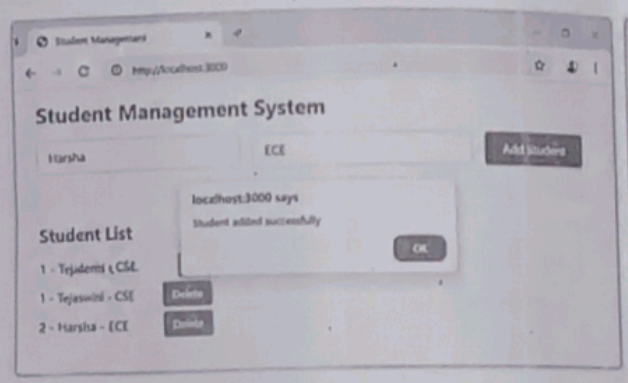
2. Initial Output in Browser (no students)



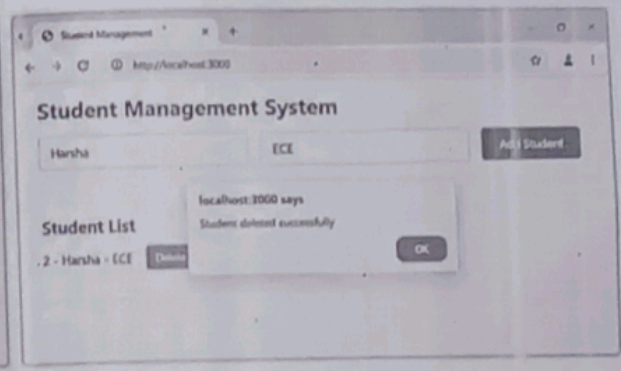
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 07

07!

Create a functional component and class component using react

Aim:- To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

Functional Component:-

A functional component is a javascript function that returns jsx to display the user interface of use react hooks such as usestate to manage state and update the webpage dynamically.

Program:-

```
import React, { useState } from
'react';
function StudentFunctional() {
    const [name, setName] = useState("vishnu vardhan");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("vishnu");
    };
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setCount(count + 1);
};
return (
  <div style={{
    border: "2px solid blue",
    padding: "20px",
    margin: "20px",
    borderRadius: "10px",
    textAlign: "center"
  }}>
<h1>Student Details</h1>
<h2>Functional Component</h2>
<p><b>Name:</b>{name}</p>
<p><b>Course:</b>{course}</p>
<p><b>Button Clicked:</b>{count}</p>
<button onClick={changeName}>changeName</button>
<button onClick={changeCourse}>changeCourse</button>
<button onClick={incrementCount}>Increment</button>
  </div>
);
```

output:- When the user clicks the buttons, the name, course and count changes dynamically without any reloading the web page

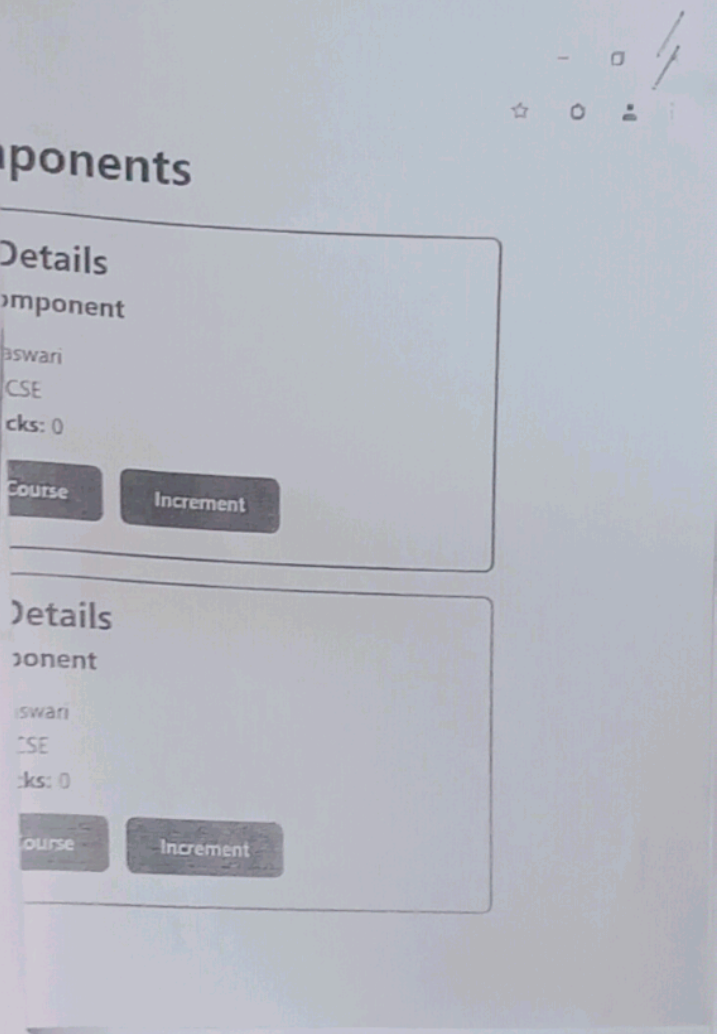
- class component:

⇒ A class component is an ES6 javascript class that extends React Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React, { Component } from 'react';

class StudentClass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name : 'vishnu vardhan',
            course : 'computer science',
            count : 0
        };
    }
changeName = () => {
        this.setState({
            name : 'vishnu'
        });
    };
changeCourse = () => {
        this.setState({
            course : 'csm'
        }); });
```




```
Change Course </button>
<button onClick = {this.increment.count}>
    Increment </button>
</div>
```

8. Implement http methods using Json Server

Aim: To implement HTTP methods such as GET, POST, PUT, PATCH, and DELETE using Json server to perform CRUD (Create, Read, Update, Delete) operations on student records

Theory: Json server is a tool used to create a fake Rest API using a json file. It allows us to perform HTTP operations without developing a backend server from scratch

HTTP methods - without developing a backend server from scratch

HTTP methods are used to communicate btw a client and a server

- GET - Retrieves data from the server
- POST - adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record from the server

a. Create a Json server

Step 1: Initialize the Project

Open the vscode terminal and execute the following commands:

```
Mkdir Jsonserver
cd Jsonserver
npm init -y
npm install json server
```

Step-2: Create db.json

Create a file named db.json and add the following student data.

```
{"students": [
  {
    "id": "1",
    "name": "vishnu",
    "course": "cse",
    "marks": "90"
  },
  {
    "id": "2",
    "name": "ram",
    "course": "cse",
    "mark": "100"
  }
  {
    "id": "1",
    "name": "manjula",
    "course": "cse",
    "marks": 100
  }
```

Step3:- Start the Json server

for the current Json server version, execute:

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods.

a. GET method: The GET method is used to retrieve student records from Json server

Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

output:

B. POST method: The POST method is used to insert a new student record into the server.

Code:

```
fetch('http://localhost:3000/students', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'nithin',
        course: 'CSM',
        marks: '695'
    })
})
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

C. PUT method: The PUT method is used to replace an existing student record with updated data

```
code: fetch ('http://localhost:3000/students/1', {
        method: 'PUT',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
          name: 'vishnu',
          course: 'CSM',
          marks: 70
        })
      })
      .then(response => response.json())
      .then(data => { console.log(data); })
      .catch(error => console.log(error));
```

User output:-

D. PATCH method: The PATCH method is used to update only selected field of an existing student record

```
code: fetch ('http://localhost:3000/students/2', {
        Method: 'PATCH',
        headers {
          'Content-Type': 'application/json'
        }
        body. JSON.stringify({
          marks: 99
        })
      })
      .then(response => response.json())
      .then(data => {
          console.log(data);
      .Then(error => { console.log(error); });
```

output:

- After executing the Post, Put, Patch, and delete operations in sequence the remaining student will be

output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS E:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

1. Terminal Output

MongoDB CRUD API

Server is running

2. Chrome Browser (Home Page)

3. Create (POST) – Add Student

4. Read (GET) – Get All Students

5. Read (GET) – Get Student by ID

6. Update (PUT) – Update Student

7. Delete (DELETE) – Remove Student

8. MongoDB Compass – Database View

Experiment-9

a. Develop CRUD operations using mongodb

Aim:- to develop a web application using node.js, express.js and mongoDB to perform CRUD operations on student records.

1. installation and Project setup:

```
mkdir mongoDB_CRUD
cd mongoDB_CRUD
npm init -y
npm install express mongoose
```

2. Program server.js:-

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const port = 3000;
mongoose.connect('mongodb://127.0.0.1:27017/crudDB')
.then(() => console.log("mongoDB Connected"))
.catch(err => console.log(err));
// Create student
const StudentSchema = new mongoose
.Schema({
    name: {
        type: String
        required: true
    }
    email: {
        type: String,
        required: true
    },
    marks: {
        type: number
        required: true
    }
});
// Create student model
const Student =
    mongoose.model("Student", StudentSchema);
// Create Add
app.post('/Students', async(req, res).
    =>{
    const Student = new Student(req.body);
    const saveStudent = await Student.save();
    res.status(201).json(saveStudent);
} catch (error){
    res.status(400).json({ message: error.
        message});
});
// Read Get all students
app.get('/students', async(req, res) => {
    try{
    const Student = await Student.find();
    res.status(200).json(message:
```

a.

```
});

// Read : Get student by id

app.get('/students/:id', async(req,res)=>
{
try{
const student = await Student.findById(req.params.id);

if (!student){
return res.status(404).json({
message: "student not found"
});
}

// update
app.put('/student/:id', async(req,res)=>{
try{
const student = await Student.findById and

update(
req.params.id
req.body
{new: true, runValidators: true
});

res.status(200).json(student);
}catch(error){
res.status(400).json({message: error.message});
}
});

// Delete:
app.delete('/student/:id', async(req,res)=>{
try{
const student = await Student.findById
and Delete(req.params.id);
if(!student){
return res.status(404).json({
message: "student not found"
});
}
res.status(200).json({
message: "student deleted successfully"
});
}
catch(error){
res.status(400).json({message: error.message});
}
});
```